Python Programming Exercise

Bioinformatics Algorithms

Class Aims

- In this class we are going to write a small Python script that reads a file, performs Euclidean Distance and outputs the result.
- But we are also going to introduce some programming concepts that will be useful for later weeks
  - Writing simple tests- how do we find out when things go wrong?
  - Thinking of code as an application

Testing Code

- There are many libraries that support testing of code (see links below)
- But when you start coding in Python the most important thing to do is test *somehow*

- You might already test
  - When you develop code eg using the PyCharm debugger
  - When you run code eg to avoid the wrong file being loaded

- However, it is also very important to consider how you also handle situations when the code shows no errors in the IDE, runs fine but generates the wrong answer!

- How do we know the code is behaving as expected? We need to run tests to determine this

- https://docs.python.org/3/library/test.html
- https://docs.python.org/3/library/unittest.html#module-unittest

Testing Code

- We always need to know the *truth*- what is the expected outcome?
- The truth can be tricky to obtain
  - We can get this from theory, from manual calculations or from other code
  - We can even store results code we ran previously and check for consistency
- If we write some sorts of test routines
  - These can be run automatically (eg at install time) to make sure the programme works as expected.
  - If the programme is complex, has lots of dependencies and runs on different platforms, exhaustive tests will be needed to complete installation.

Testing Code – Large Language Models

- LLMs such as ELM can help you write code, they can also generate code. But they do not test code for correctness, and they do not know the purpose to which you will put the code.

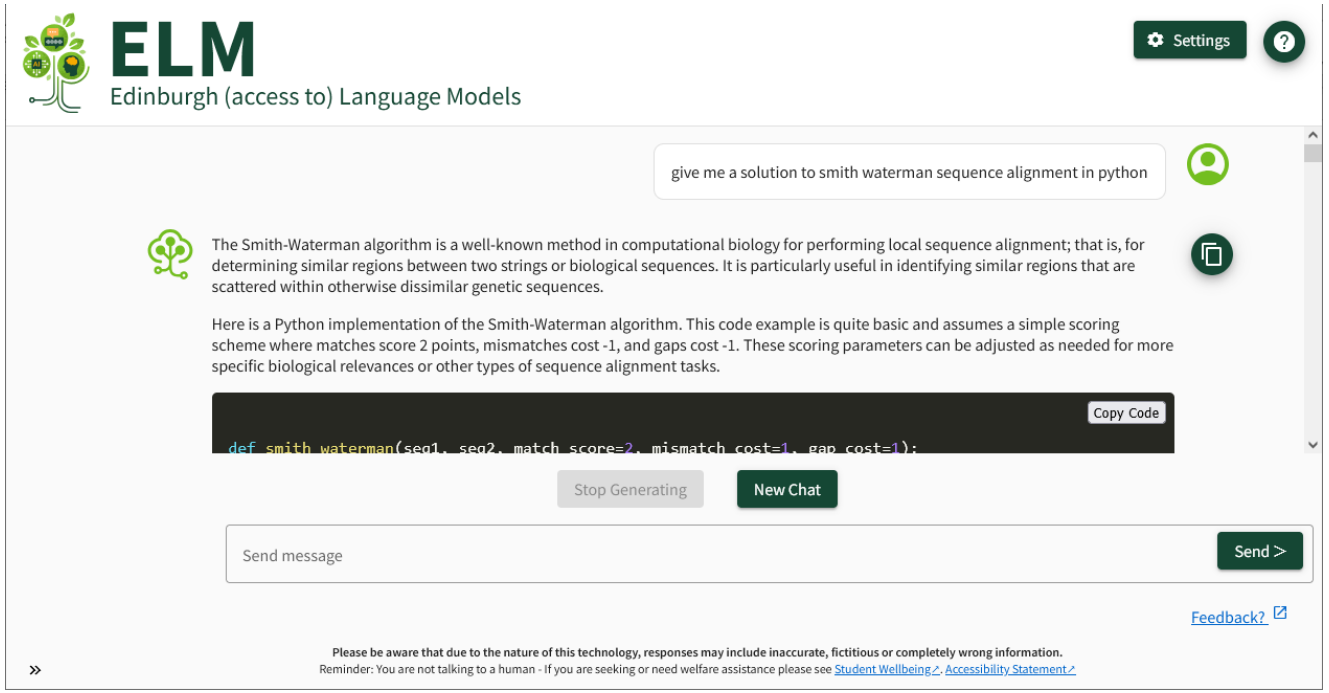
- If you use LLMs for code generation (when appropriate), you really need to test the code!!

- If you find the code does not work, and you do not understand the functioning of the code, it can be *really* difficult to fix!

Using ELM to Generate Code ELM

https://elm.edina.ac.uk/elm/elm

- The code in this course is not written using LLMs!
- However, we can use LLMs as a source of code to try and test. Here we use ELM but many LLMs generate code.
- I simply went to the ELM system, logged in and asked
  - “give me a solution to smith waterman sequence alignment in python”



- Lots of algorithms classes start with this algorithm- and so ELM has a solution
- I can just copy the code

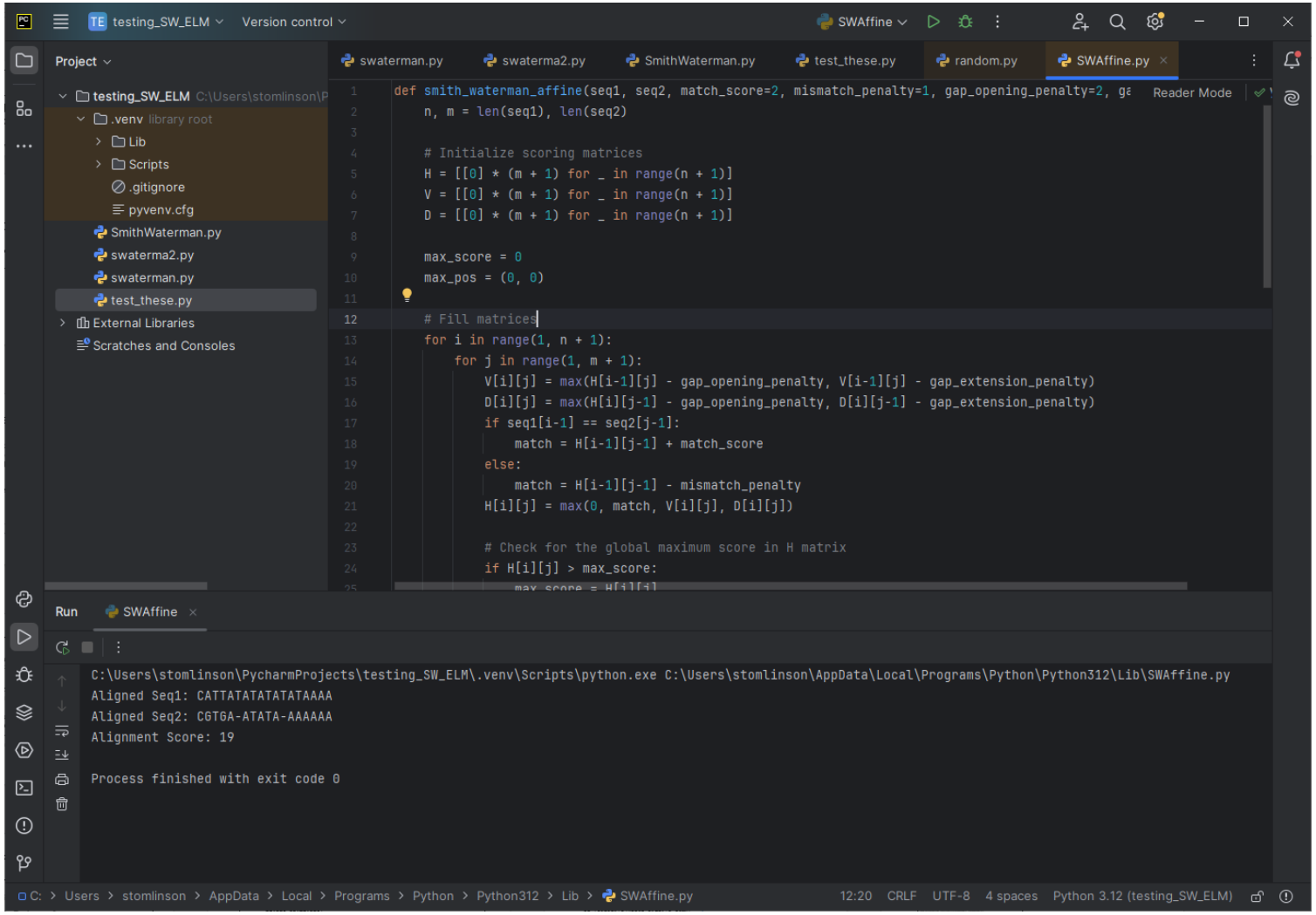
ELM

- I also asked “This doesn't get the right answer- can you fix it?”
- Actually the answer was fine but ELM did not know this and responded “Certainly! Let's correct the Smith-Waterman algorithm implementation for local sequence…”
- So now I have three solutions to pairwise alignment- but are they correct? Do they produce the correct score and alignment?

Comparing to Existing Code

- If the SW algorithms work as expected they should get the correct answer against reference software- such as Water.
- They should also all give the same results when compared to each other!
- In the class we compare to the two ELM SW implementations against the one developed in class (as this has been checked against Water and against manually calculated results).
- Because each software generates one top scoring alignment, but not the same top scoring alignment- the approach is just to compare scores…
- The ELM implementations much more succinct than the class code but they do not generate the DP matrix or a proper alignment.

Within Pycharm



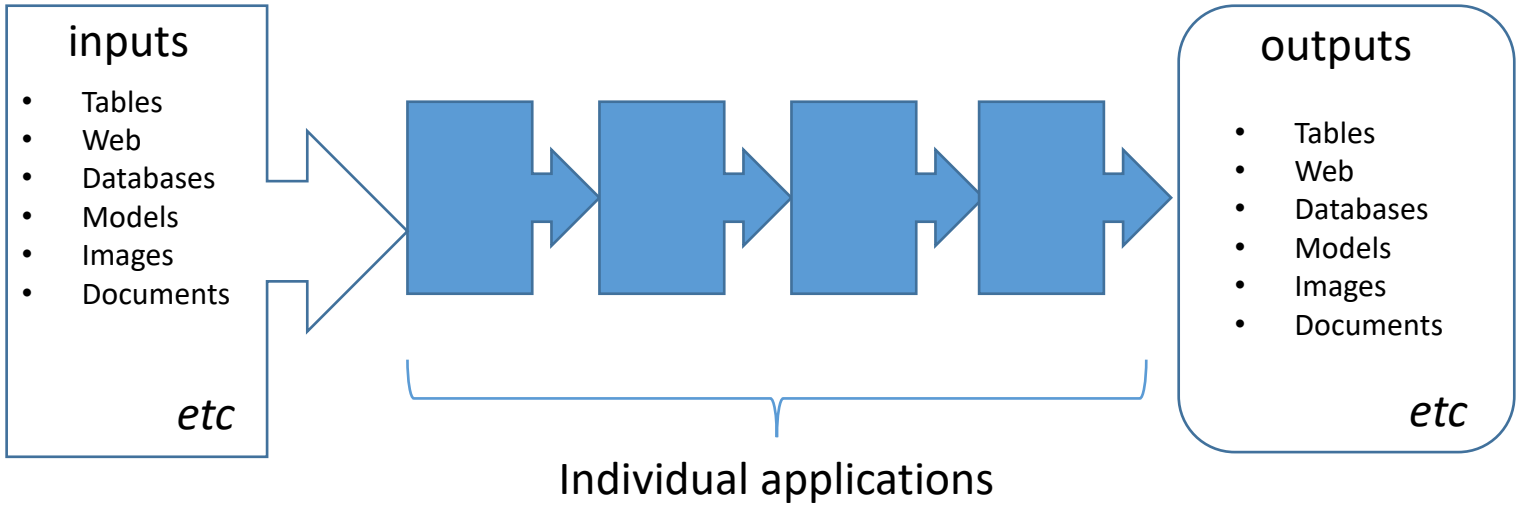
- I created a new project
- I put all the ELM SW implementations into PyCharm (in individual files)
- I hacked these a little but so they returned the score and so the class code did not print out alignments and the DP table
- Then I wrote a test script called test_these that could then test all the scripts

```python

#Set this to run the cycles
def regress_my_code(cycles =40):  1 usage

    for i in range(0,cycles):
        len_seq1 =0
        len_seq2 =0

        while len_seq1 <10 or len_seq2 <10:
            len_seq1 = random.randrange( start: 0, stop: 40)
            len_seq2 = random.randrange( start: 0, stop: 40)

        print("len seq1:", len_seq1)
        print("len seq2:", len_seq2)

        seq1=[]
        seq2=[]
        for ii in range(0,len_seq1):
            seq1.append(random.choice(dict))
        for j in range(0, len_seq2):
            seq2.append(random.choice(dict))
        seq1 = ''.join(seq1)
        seq2 = ''.join(seq2)


        mat = int(random.choice(m_scores))
        mis = int(random.choice(mm_scores))
        gap = int(random.choice(g_scores))

        aligned_seq1, aligned_seq2, score1 = elm1.smith_waterman(seq1, seq2, mat, mis, gap)
        aligned_seq1, aligned_seq2, score2 = elm2.smith_waterman2(seq1, seq2,mat,mis,gap)
        score3 = SRT.perform_smith_waterman(seq1, seq2,mat,mis * -1,gap *-1)

        if  score2 !=score3 or score1 !=score2:

           print(" seq1: ",seq1)
           print(" seq2: ",seq2)
```

- Here rather than test one pair of sequences I generate sequences each time the code loops. This generated sequence is random. Basically I write all the valid bases in an object called “dict”- [this is actually a list not a dictionary!]
- Random sampling from “dict” gives a list of valid bases that can be assembled into a sequence
- Just for fun I make these each a random size
- I also use a similar approach to set alignment parameter
- Then I run each aligner on the random seq and with the settings and check they all get the same result.
- If I do this within a loop and all the score agree then fine, but if they do not agree then there is a problem.

- A similar approach to generating random seqs is used for simulations…it to obtain the parameters for Blast so p-values and e-values can be calculated

Code as an Application

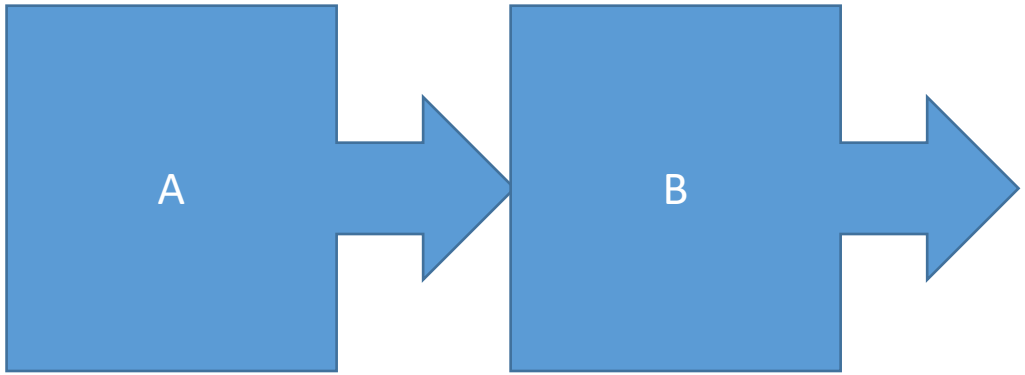


- Workflows typically orchestrate lot of individual applications feeding the output of one component into another component as input
- Individual applications take input and output and provide a step along the way

This is a simplification, but it is a useful way to think about the different ways we use Python within the teaching programmes.

From the Python Primer

Code as an Application



- Individual components such as A or B take input and generate output
- If we enable Python code to pick up options from the command line, we have an easy way of feeding input from one components into another. In essence A and B are applications that are part of a larger pipeline.

In Practice

- We need to consider command line interfaces and perhaps also loading options from files. One common approach is to load the specific user commands from the command line and defaults from a file.
- We also need to consider the flow of execution in our application, especially when using functions and multiple files. We can run any Python files as an executable within the IDE. So we have multiple entry points for the application.. we really should have one
- Finally we need to consider how variables work across files.

ConfigParser

https://wiki.python.org/moin/ConfigParserExamples

Basic configparser usage

These are some examples on using ConfigParser, assuming the following INI file...

```
[SectionOne]
Status: Single
Name: Derek
Value: Yes
Age: 30
Single: True

[SectionTwo]
FavoriteColor = Green
[SectionThree]
FamilyName: Johnson

[Others]
Route: 66
```

```
>>> import ConfigParser
>>> Config = ConfigParser.ConfigParser()
>>> Config
<ConfigParser.ConfigParser instance at 0x00BA9B20>
>>> Config.read("c:\\tomorrow.ini")
['c:\\tomorrow.ini']
>>> Config.sections()
['Others', 'SectionThree', 'SectionOne', 'SectionTwo']
>>>
```

Explanation: We first import the configparser, tell it to read the file, and get a listing of the sections. Sections are listed in square brackets []

- A useful tool for reading and writing configs
- Settings can be stored in a file and this file parsed on start
- Settings can be read and written to the file

Code File Imports

mymain_files.py

```
import mypython_files as mpf
import mypython_files2 as mpf2

mpf.myfunction()

mpf.mystring ="changed String"
mpf2.mystring ="changed String2"

print("MPF string",mpf.mystring)

print("MPF2 string", mpf2.mystring)
```

Run this file

mypython_files.py

```
print("Hello World")

mystring="a String"

def myfunction():
    print("Internal Function Call")

myfunction()
```

mypython_files2.py

```
import mypython_files as mpf
```

Output

Hello World
Internal Function Call
Internal Function Call
MPF string changed String
MPF2 string changed String2

- Here note that the import into mypython_files give access to a new copy of the string from the import
- We can access mpf2.mystring even though the code for this is in the imported file

Class Practical

- Perform an analysis to find the index of the cloest gene in terms of distance in the gene table compared to an input gene
- Plot the gene generated from this analysis

Example:

Given a set of gene measurements across an experiment find the closest pair of genes by Euclidian distance

$$\mathrm{d}(\mathbf{p},\mathbf{q}) = \mathrm{d}(\mathbf{q},\mathbf{p}) = \sqrt{(q_1-p_1)^2+(q_2-p_2)^2+\cdots+(q_n-p_n)^2} = \sqrt{\sum_{i=1}^{n}(q_i-p_i)^2}.$$

Gene/Person	**Person1**	**Person2**	**Person3**	**Person4**	**Person5**	**Person6**
1	56	10	12	12	34	45
2	56	10	12	12	34	45
3	54	20	23	9	21	50
4	43	20	34	8	23	60
5	44	20	54	4	27	40
6	41	20	2	9	29	20

In this algorithm if Gene 3 is p and Gene 4 is q then p_1 =54 and q_1 =43

- Each Person measurement in the columns can be represented as a dimension in space
- Compare gene 1-6 across each of Person1 to Person6 to calculate a triangular distance matrix that compares the correlation of each gene to each other gene
- Find the smallest pair of genes (1 and 2 in this case)

Example

- Compare gene 1-6 to each of Person 1 to Person 6 to calculate a triangular distance matrix
- Find the smallest pair of genes (1 and 2 in this case)

- **Brute force algorithm would be**

1. The data would be in a 2D array [matrix] data structure
2 Traverse each row element and then
3 Compare this row to each of the other rows,
4 Save the Euclidian result in a distance matrix
5 Find the smallest distance by traversing the distance matrix again

- Algorithm complexity and comments
  - The core loop comparing each row to all other rows is most costly and will perform N2 comparisons
  - In practice stage 4 can be performed during 2 and 3 saving just the current most similar pair
  - Only half the table needs calculating as d(1,2) is the same as d(2,1)
  - Overall the complexity is O(N2)
  - We will revisit this problem later with a faster algorithm!

Implementation in Python- setup plotting

```
#pip install plotly==4.5.4 (use the Pycharm installer if plot.ly is not available
#see https://plot.ly/python-api-reference/generated/plotly.graph_objects.Figure.html
#see plot.ly/
import plotly.graph_objects as  go
import plotly.offline  as  offline
import csv
import math
```

Plotting

```
#Simple plotting function
def plotme(xx,yy):
    fig = go.Figure(
       data=go.Pointcloud(x=xx, y=yy),
       layout=go.Layout(
          title=go.layout.Title(text="A Scatter Plot"))
    )
    offline.plot(fig, filename='file.html‘,auto_open=False)
```

- This is plotly, we use a variety of other plotting tools in class.

Processing Pipeline

```
#Test
dd=euclidean_distance([4,5,6,7],[4,5,12,7])

#Load files-small file for testing, big
file for final search
ed=loadafile("test_bigfile.csv")
#ed=loadafile("test_file1.csv")

#Find most correlated pair
ee=findmostcorrelated(ed)

#print simple results
print("result",dd)
print("result2",ee[0],ee[1],ee[2])

#Plot the winning correlation
plotme(ed[ee[0]],ed[ee[1]])
```

two test files

Missing code that needs replacing…

```
#load a file- this will contain comma delimited values for correlation
def loadafile(flename):

   mytable =[]
#missing code
   return mytable


#calculate Euclidean distance
def euclidean_distance(element1, element2):

   if(len(element1)!=len(element2)):
        print("values wrong length....")
        return -1

   d=0

  #missing code
   return d


#find the least distant pair of indexes using this method
def findClosest(mytable):
   corr=0.0
   val1=-1
   val2=-1

   #missing code
   return val1, val2,corr
```

hint

```
with open(flename) as values:
      g_reader = csv.reader(values, delimiter=',')
      for value in g_reader:
```

You could also use Pandas

- The answer has been provided in case you are stuck!

Missing code that needs replacing…

```
#load a file- this will contain comma delimited values for correlation
def loadafile(flename):

   mytable =[]
#missing code
   return mytable


#calculate Euclidean distance
def euclidean_distance(element1, element2):

   if(len(element1)!=len(element2)):
        print("values wrong length....")
        return -1

   d=0

  #missing code
   return d


#find the most correlated pair of indexes using this method
def findmostcorrelated(mytable):
   corr=0.0
   val1=-1
   val2=-1

   #missing code
   return val1, val2,corr
```

Write a function to load the data for parsing from the table

Write a function to calculate Euclidean distance

This is a distance value

Compare all-against-all and find the closest correlated
Return the row indexes of this pair and the value

Note the Code Organisation

- The file is interpreted line by line from top to bottom
- The header at top of the file provides a human-readable summary
- Libraries are loaded
- Functions Defined
- Then assemble the final programme from the defined functions and various inputs
- The libraries are loaded, functions called and then the Python executing environment has the required components for the rest of the programme
- More complete code might have full details in the header, build a command line so the inputs can be picked up from this and then this can be used to perform the final calculations.
- Code testing and handling of errors is important
  - We should test the code as we develop it against known “correct” outputs
  - If our programme can load different files, or take different inputs (eg from a command line) then it is very important to check that these inputs are valid. Handling of errors at runtime is needed for this.
  - *We only have simplified error handling in this code!!*

Read the File A Row At a Time

```
#load a file- this will contain comma delimited
values for correlation
def loadafile(flename):

    mytable =[]
#missing code
    return mytable
```

```
 mytable =[]

    with open(flename) as values:
        g_reader = csv.reader(values,
delimiter=',')
        for value in g_reader:
            test =list(value)
            test = [float(i) for i in test]

            mytable.append(test)
    return mytable
```

- The file loaded is a matrix of comma separated values
- Read the file one row at a time and store row-wise in a Python list
- Note how a csv.reader is defined and this use to loop over the file contents one row at a time
- Each row is then stored in the [] Python list after being converted to float values

Generate Euclidean Distance

```
• #calculate Euclidean distance
• def euclidean_distance(element1,
  element2):

•     if(len(element1)!=len(element2)):
•           print("values wrong
  length....")
•           return -1

•     d=0

•    #missing code
•     return d
```

```
def euclidean_distance(element1, element2):

    if(len(element1)!=len(element2)):
          print("values wrong length....")
          return -2

    d=0

    for x in range(0, len(element1)):
            d=d+(element1[x]-element2[x])**2
    d=math.sqrt(d)
    return d
```

- Note here I check that the vectors (really lists) element1 and element2 , representing two rows, are the same length
- We do this to ensure that we do not try to index outside the size of element2
- By designing our code in this way, we can apply the `euclidean_distance` function on any pair of rows

Find the Least Distant

```
#find the least distant pair of indexes
using this method
def findclosest(mytable):
    dist=0.0
    val1=-1
    val2=-1

    #missing code
    return val1, val2,dist
```

```
def findclosest(mytable):
    dist=0.0
    val1=-1
    val2=-1

    for x in range(0,len(mytable)):
            for y in range(x+1,len(mytable)):
                mydist=euclidean_distance(mytable[x],mytable[y])

                if(val1==-1):
                    dist = mydist
                    val1 = x
                    val2 = y
                    continue

                if mydist<dist:
                    dist=mydist
                    val1=x
                    val2=y
    return val1, val2,dist
```

- So this actually goes through all the rows in the table and compares to all the rest
- For each pair of rows, Euclidean distance is calculated
- It just saves the lowest value and the rows that generated this. In the end it will return the lowest distance
- Really this function has a confusing name as it is finding the pair of rows that have the closest distance to each other- but essentially these are the most correlated as well!!

Extra Coding Tips!!!

- Try working out the answer without looking at these!!
- There are a number of different ways in Python to achieve each task and obviously one would just be to call library code implementing each algorithm
- The aim here is to code from first principles…
- The code implements a simple pipeline for processing the input data and generating a plot.

Questions

Correlation

- What are the indexes of the closest distance pair and their distance?

- How would you extend this programme to build a complete correlation/distance matrix??